HPCA 2027 Submission #NaN
Confidential Draft
Do NOT Distribute!!

QArchGauge: Architecture-Guided Modeling of Practical Quantum Advantage

Abstract*—Practical quantum advantage is an architecture question: a quantum-circuit application must beat a strong native HPC implementation on the same input, at the same quality, after all application overheads are included. However, simulator throughput and hardware speedup claims do not tell architects which resource should improve: logical gates, shot parallelism, decoder latency, host–QPU I/O, hybrid scheduling, native baselines, or algorithmic quality. This paper presents QArchGauge, an architecture-guided modeling framework that converts same-input quantum-circuit measurements into practical QPU requirements on a leadership-scale HPC system. QArchGauge executes native implementations and quantum-circuit formulations with shared inputs and quality targets, then combines native runtime, circuit cost, circuit structure, repeated-evaluation demand, and quality gap to project a break-even frontier over pipelined execution, shot throughput, quality recovery, and serial host/control overhead. This design treats `cuQuantum` simulation as exact, state-vector-verifiable instrumentation for estimating future-system requirements, not as evidence of current or deployment-scale quantum advantage. Our evaluation covers 160 cases from `sklearn digits` and a 3,552-case practical suite across ML, Chem, Opt., and Sim. on up to 256 GPUs, plus a 104-case OpenFermion/PySCF Chem coverage gate. The digits calibration requires median speedups of 421.9× for quantum kernels and 64.9× for QNN/VQC; practical-suite medians span 3,071.0× for Sim. to 287,045.6× for Opt. Under the default lower-bound projection at 90% quality-gap recovery and no recovery-cost penalty, 10^4× moves 54.9% of Sim. cases into the advantage region and 10^5× moves 57.1% of Chem cases, while conservative shot, host-I/O, and queue/control sensitivity and any physical recovery overhead remove much of this coverage. These results turn vague quantum-speedup claims into workload-specific architecture targets for quality, batching, host/control movement, and HPC–quantum co-scheduling.

I. Introduction

Quantum computing is often described through broad application promises in machine learning, chemistry, optimization, and scientific simulation [1]–[3]. However, practical quantum advantage requires more than executing a quantum circuit. A quantum-circuit application must beat a strong native HPC implementation on the same input, at the same quality, after encoding, circuit generation, repeated evaluations, measurement, optimizer steps, and postprocessing are included. Thus, for computer architects, quantum advantage is not only an algorithmic claim but a resource-targeting problem: future systems must reveal whether the limiting resource is logical gate latency, two-qubit fidelity, shot throughput, readout/control latency, controller-side memory movement, native-baseline strength, or algorithmic quality.

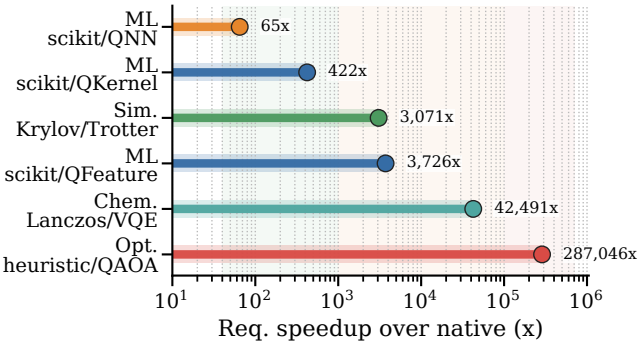


Fig. 1: Required speedup for each native/quantum pair. ML denotes classification, Chem denotes molecular-energy estimation, Opt. denotes combinatorial optimization, and Sim. denotes Hamiltonian/scientific simulation.

To study this problem before large fault-tolerant machines are available, HPC systems provide a practical measurement platform. Libraries such as `cuQuantum` execute quantum circuits on GPUs, while prior systems such as ScaleQsim and AURORA-Q improve circuit-simulation scalability on leadership-scale systems [4]–[6]. These simulators are essential for generating evidence, but simulator scalability is not equivalent to application-level quantum advantage. A simulator can improve the circuit kernel while the full quantum path remains slower than a native HPC application. Therefore, an advantage study must measure the native deadline, the quantum-circuit application cost, the quality gap, the repeated-evaluation structure, and the projected hardware stack together.

Figure 1 shows the comparison target of QArchGauge. A circuit simulator is only one component of the quantum path. The full quantum path also includes application stages that do not disappear when the circuit kernel becomes faster. Three points follow. First, simulator speed is not application speed: measuring only `cuQuantum` time can hide encoding, measurement, optimization, and classical control. Second, native baselines are an architecture constraint because native HPC/ML applications already use mature software stacks and efficient kernels. Third, advantage is a design-space boundary: the goal is to invert today's measurement and compute the required quantum gate time, shot throughput, error overhead, and quality recovery for future hardware. That boundary assigns pressure to concrete architecture resources: gate pipelines,

TABLE I: Comparison with prior work across five capabilities: Native Deadline (ND), Quality Target (QT), Application Cost (AC), Hardware Projection (HP), and Architecture Target (AT).

Study	Target	ND	QT	AC	HP	AT
`cuQuantum` [4]	GPU circuit kernels			partial		
ScaleQsim/ AURORA-Q [5], [6]	HPC circuit sim.			partial		
SuperMarQ [2]	HW benchmark		✓	partial	partial	partial
QASMBench/ MQT [7], [8]	Circuit benchmark		partial	partial		partial
QED-C [3]	App. benchmark		✓	partial	partial	partial
Azure/QEC studies [9], [11]	FT resource model				✓	partial
HPC–Q runtimes [13], [14]	Hybrid integration			partial	partial	partial
Native baselines [15]	Classical apps	✓	partial			
QArchGauge	**Advantage model**	✓	✓	✓	✓	✓

shot lanes, decoder/control latency, host–QPU data movement, quality-preserving encodings, and native-baseline strength.

As shown in the figure, the required speedup is highly workload-dependent. Controlled ML circuit paths require 65× for QNN/VQC and 422× for quantum kernels. Practical Sim. and ML workloads require thousands of times faster quantum execution, while Chem and Opt. reach tens of thousands to hundreds of thousands of times. These results highlight a key limitation: measuring one circuit kernel or one narrow workload does not identify the architecture requirement. ML stresses data encoding, kernel construction, and hybrid training loops. Chem stresses repeated energy evaluation and ansatz quality. Opt. stresses repeated search over approximate objectives. Sim. stresses Hamiltonian evolution depth and operator structure. The measured cases are exact, state-vector-verifiable proxy records rather than deployment-scale claims; they are useful because every native deadline, circuit record, quality gap, and projection knob can be audited before larger tensor, approximate, or hardware-backed campaigns are attempted. Therefore, QArchGauge reports an advantage frontier rather than a single yes/no answer.

Many previous studies, as summarized in Table I, focus on one part of this modeling stack. HPC quantum simulators and GPU libraries improve circuit execution on classical systems [4]–[6]. Benchmark suites such as SuperMarQ, QASMBench, MQT Bench, and QED-C improve workload coverage, metrics, and reproducibility [2], [3], [7], [8]. Resource-estimation tools and fault-tolerance studies expose physical-qubit, code-distance, cycle-time, decoder, and layout assumptions [9]–[12]. HPC–quantum integration studies examine how QPUs attach to scientific computing ecosystems [13], [14]. However, these studies do not jointly connect native deadlines, application-level quantum costs, quality targets, hardware projections, and architecture target diagnosis.

QArchGauge distinguishes itself from prior work by treating quantum advantage as an application-level architecture boundary. Instead of only accelerating circuit simulation, defining circuit benchmarks, or estimating fault-tolerant resources, QArchGauge measures native and quantum-circuit paths on shared inputs and then projects which future hardware component must improve. This connects simulator evidence to architecture decisions: faster logical operations, batched shots, lower decoder/control latency, quality-preserving encodings, stronger native baselines, or tighter HPC–QPU integration.

Key Idea. QArchGauge treats practical quantum advantage as an architecture design-space boundary. The threshold is the point where the projected quantum-circuit path becomes faster than the measured native HPC/ML path under the same workload and quality target. For each case, QArchGauge records native time, circuit-application time, quality gap, gate/measurement counts, repeated-evaluation structure, and Slurm/GPU provenance. It then inverts those records into an architecture target map: whether the useful improvement is logical-operation speed, shot parallelism, decoder/control latency, measurement aggregation, algorithmic quality recovery, or a stronger native baseline. The output is a hardware hint, not one global speedup number.

This paper makes the following contributions through QArchGauge, an architecture-guided quantum-advantage modeling and analysis study for HPC systems. Specifically, QArchGauge (1) defines practical quantum advantage as a same-input, same-quality break-even condition between a native HPC path and a quantum-circuit application path, (2) builds an auditable pipeline that records runtime breakdowns, quality, circuit metadata, GPU metadata, Slurm metadata, and repeat-timing evidence, (3) evaluates controlled ML and practical ML, Chem, Opt., and Sim. workloads on Perlmutter up to 256 GPUs, and (4) converts measured circuit costs into required execution speed, shot throughput, serial-control reduction, and quality-gap recovery. Our evaluation covers 160 controlled digits cases and a 3,552-case practical suite. The results show that required speedups vary from 65× for QNN/VQC to 287,046× for Opt., and that future quantum systems need workload-specific architecture targets rather than one global speedup number.

II. Background

A. Quantum-Circuit Application Paths

Quantum-circuit applications wrap a classical input or scientific instance into a circuit, execute that circuit many times, and return samples, observables, kernel entries, or energy estimates to a classical outer loop. This differs from a stand-alone circuit benchmark. The circuit kernel is only one stage in an application path, and the application can fail to be useful even when the circuit itself becomes faster.

ML Circuit Paths. Quantum machine-learning workloads commonly use two circuit paths. In a QNN/VQC path, classical features are encoded into a parameterized quantum circuit, and a classical optimizer updates circuit parameters using repeated circuit evaluations. In a quantum-kernel path, a quantum feature map transforms pairs of input samples into kernel values; many circuit evaluations are then assembled into a kernel matrix used by a classical classifier. Thus, QNN/VQC stresses hybrid-loop latency and parameter quality, while QKernel stresses repeated feature-map evaluations and kernel-matrix construction.

Practical Application Families. Beyond ML, this paper studies Chem, Opt., and Sim. circuit paths because they expose different architecture limits. Chemistry workloads often use VQE-style paths: a molecular Hamiltonian is mapped to Pauli terms, an ansatz prepares a trial state, repeated measurements estimate energy, and a classical optimizer searches for low-energy parameters. Optimization workloads often use QAOA-style paths: a graph or objective is encoded into phase-separation gates, mixer gates explore bitstrings, and repeated sampling estimates objective quality. Hamiltonian-simulation workloads use Trotterized or related unitary-evolution paths, where Hamiltonian terms become a sequence of gates. These paths stress different resources: energy-measurement repetition for Chem, search quality for Opt., and evolution depth for Sim.

Repeated Execution. The key systems implication is repetition. A single circuit execution is rarely the full application. Training uses samples, epochs, optimizer steps, and candidate parameters. Kernel methods use many sample pairs. VQE uses many Pauli-term measurements and ansatz updates. QAOA uses many graph instances, parameter settings, and samples. Therefore, application cost must include encoding, circuit construction, repeated execution, measurement aggregation, optimizer or classifier work, and postprocessing. Reporting only one circuit-kernel time hides the repeated-evaluation structure that dominates many practical workloads.

Terminology. This paper uses "quantum path" to mean the complete quantum-circuit application path, not only one circuit kernel. It uses "native path" to mean the non-quantum HPC/ML implementation for the same input. It uses "advantage region" only when projected runtime and application quality both satisfy the workload target.

B. Native Baselines and Threshold Model

Practical quantum advantage is a same-input comparison against the native application, not against another simulator. The native path directly solves the task on CPUs or GPUs using classical numerical, ML, chemistry, optimization, or simulation methods. The quantum path solves the same task through a quantum-circuit formulation. QArchGauge uses the native path as a deadline: the projected quantum path must finish before this deadline while meeting the same quality target.

Leadership-scale GPU Platform and cuQuantum. The role of the leadership GPU system is measurement and evidence generation. `cuQuantum` executes the circuit application path and exposes circuit structure, repeated-evaluation cost, quality, and simulator overhead. The later QPU projection replaces this measurement stack with explicit logical-operation, shot, decode, control, and queue terms.

Native Deadline. The native deadline is the fastest valid native implementation for the same input and quality metric. This definition is important because native software is a moving architecture target: adding a stronger classical solver, optimized GPU kernel, or production ML baseline can move the break-even point. For HPCA-style evaluation, the native path is therefore not a weak reference implementation; it is the performance target that future quantum hardware must beat.

Two Runtime Axes. QArchGauge keeps two runtime axes separate. The measured axis is $T_{\mathrm{qsim}}/T_{\mathrm{native}}$: how much slower the full quantum-circuit application path is when executed today through the `cuQuantum` measurement stack. The projected axis is $T_{\mathrm{qhw}}/T_{\mathrm{native}}$: how the same recorded circuit depths, measurements, shots, repeated evaluations, and quality gaps would behave under an explicit future-hardware model. A workload can therefore have a large measured simulator slowdown and still look runtime-attractive under an optimistic future QPU model if it has little repetition and maps directly to unitary evolution. Such a point is not a current advantage claim; it is a conditional architecture target.

Break-even Condition. Runtime alone is insufficient. A quantum-circuit path that is faster but has lower accuracy, worse energy, worse approximation ratio, or larger observable error has not reached application-level advantage. QArchGauge uses a workload-specific quality gap ΔQ and tolerance ϵ_w. A case enters the advantage region only when

$$T_{\mathrm{qhw}} < T_{\mathrm{native}} \quad \text{and} \quad \Delta Q \leq \epsilon_w.$$

This condition does not claim that current quantum hardware wins. It defines the break-even boundary under which a future quantum system would become competitive for the measured input.

Threshold and Frontier. The practical quantum-advantage threshold is the amount of projected improvement needed for a quantum-circuit application path to enter the advantage region. Because each workload has different native deadlines and quality gaps, the threshold is not one global speedup. QArchGauge therefore reports a frontier over execution speed, shot throughput, non-kernel overhead, and quality recovery. The recovery parameter is a requirement axis, not a free mitigation result: better encodings, deeper ansatz circuits, higher Trotter order, more shots, or stronger mitigation may reduce ΔQ, but they can also increase runtime and repeated evaluations. We model this possibility with a recovery-cost multiplier in the projection model; the default plots use the optimistic lower bound and the sensitivity text explains how physical recovery costs move the frontier. A case can be speed-limited when quality is close but runtime is too high, quality-limited when faster gates do not help, or native-dominated when the classical path is already too strong.

Architecture Levers. Once runtime and quality are separated, the remaining question is architectural. Different failures point to different hardware changes, and treating them as one generic speedup hides the useful design signal.

Gate and Shot Throughput. Faster logical one- and two-qubit operations reduce circuit-depth cost. However, many applications also need many shots or repeated circuit evaluations. More devices or more shot lanes help only if readout, reset, decoding, control bandwidth, and queueing can sustain independent shots. Thus, shot parallelism is a system resource, not merely a circuit parameter.

Control, Decode, and Data Movement. Hybrid applications repeatedly exchange information between the quantum device and the classical controller. Readout, reset, decoding, measurement aggregation, optimizer feedback, and host–QPU data movement can become a serial floor even when logical gates are fast. These terms motivate near-QPU control and decode acceleration, batching, and co-scheduling with HPC resources.

Algorithmic Quality. Some failures remain even in noiseless circuit simulation. QNN/VQC may underfit or train poorly, QKernel may require too many pairwise circuits, VQE may depend on ansatz quality, QAOA may need deeper circuits or better search, and Trotter simulation may accumulate approximation error. In these cases, the architecture target is not only faster gates; it is quality-preserving encoding, ansatz/search structure, fidelity, mitigation, or a workload-specific accelerator style.

Accelerator Implications. The same threshold record can point to different future machines. General circuit workloads point toward fault-tolerant gate-model systems with strong logical operations and shot lanes. Repeated-measurement workloads point toward controller/decode acceleration and batching. Hamiltonian dynamics may benefit from analog or digital simulation engines. Energy-minimization instances that embed cleanly may motivate annealing-style specialization. QArchGauge uses HPC simulation as the measurement instrument that maps each workload to these architecture levers.

III. Design

Overview of the Framework. QArchGauge answers one architecture task: identify which future quantum-system resource must improve before a quantum-circuit application can beat the native HPC application for the same task. It is not a new simulator, compiler, or quantum algorithm. It is a measurement and projection framework that keeps the input, quality target, native baseline, quantum-circuit path, and hardware projection in one auditable record.

A. Overall Procedure

The design has three duties. First, it makes the comparison fair: the native path and quantum-circuit path use the same input and quality metric. Second, it explains the gap: the record separates native time, circuit execution, repeated evaluations, measurement, quality loss, and simulator overhead. Third, it converts the gap into a hardware target: a case should say whether the next useful improvement is faster logical operations, more shots, lower decoder/control latency, better algorithmic quality, or a different accelerator style.

These duties define the evaluation order. The controlled record supports the setup gate: every result must say which native application and which circuit application are being compared. The workload fields support the threshold gate: ML, Chem, Opt., and Sim. are not averaged into one number because each creates a different runtime-quality boundary. The Slurm and GPU fields support the scaling gate: scale-out results measure evidence-generation throughput and expose serial orchestration, not a fictional multi-GPU QPU. The quality

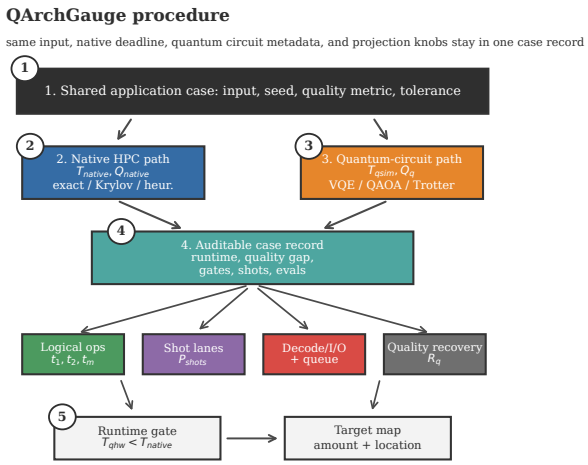


Fig. 2: Architecture and procedure of QArchGauge.

fields support the application-quality gate: a noiseless circuit path can still lose through encoding, ansatz, Trotter, search, or native-baseline strength. The projection fields support the architecture gate: the final output is an amount, a location, and an actionability test for future hardware.

B. Shared Workload Control

Shared input and quality. Figure 2 shows the core rule. Before any path runs, QArchGauge creates one configuration record with workload family, instance ID, random seed, feature transform, native candidates, circuit family, circuit depth, shot setting, and quality metric. The native path and the quantum path receive this same record and append path-specific measurements. This prevents a quantum circuit from being compared against a different split, feature dimension, molecule, graph, Hamiltonian, or quality target.

Config Record. The configuration record is the smallest unit that can support an advantage claim. It stores the instance identity, native-candidate list, quantum-circuit family, quality metric, tolerance, random seed, software versions, and Slurm/GPU context when available. The record is append-only: native execution adds native time and quality, quantum execution adds simulator time, circuit metadata, and quality, and projection adds future-hardware parameters. This structure makes later figure generation auditable because every plotted point can be traced back to one same-input record.

C. Application Path Execution

Native path. The native path executes the best implemented non-quantum method for the same input. ML includes classical classifiers plus a production-native gate with PyTorch AMP CNN/MLP and XGBoost GPU-hist on the same measured digits configurations. Chemistry uses dense exact and sparse Lanczos/eigensolver baselines, optimization uses exact enumeration when feasible plus greedy, local-search, and annealing baselines, and simulation uses dense exact and sparse Krylov-style evolution. The threshold always uses the fastest valid native candidate that reaches the best native quality

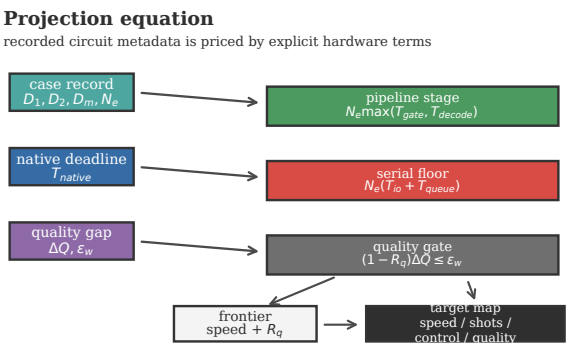


Fig. 3: Projection flow in QArchGauge. Measurements are not the final answer; they are inverted into speed, quality, shot, and native-baseline requirements.

within a small tolerance, so adding a stronger native method can only make the quantum target harder.

Quantum-circuit paths. The quantum kernel path maps the same features into a quantum feature map, simulates the circuit evaluations needed to build a kernel matrix, and passes that matrix to a classical classifier. The QNN/VQC path maps the same features into a parameterized circuit and updates trainable parameters through a classical optimizer. QArchGauge records application-level cost for both paths: encoding, circuit construction, `cuQuantum` execution, kernel or observable extraction, optimizer iterations, classifier-head time, accuracy, gate counts, shots, and repeated evaluations. This makes repeated kernel entries and hybrid optimizer loops visible instead of reporting only one fast circuit kernel.

D. Measurement and Threshold Analysis

Simulator-hardware separation. The measured `cuQuantum` runtime is not a physical quantum latency. It executes the quantum-circuit application path, produces output quality, and records circuit metadata. The projection then prices the same case under future logical-operation speed, effective shot parallelism, error/control overhead, and quality recovery. Thus Python orchestration, GPU setup time, and simulator state-vector behavior are measurement-stack evidence, not future-QPU schedule terms.

Execution model. Given the measured metadata, QArchGauge estimates projected quantum execution time with a pipelined FT-style critical path:

$$T_{\text{gate}} = \frac{N_s(D_1 t_1 + D_2 t_2 + D_m t_m)}{P^{\text{eff}}_{\text{shots}}},$$

$$T_{\text{execute}} = N_{\text{eval}}[\alpha_R \max(T_{\text{gate}}, T_{\text{decode}}) + T_{\text{io}} + T_{\text{queue}}].$$

where N_s is shot count, N_{eval} is the measured number of circuit evaluations, D_1, D_2, and D_m are one-qubit, two-qubit, and measurement depths per evaluation, t_i are projected logical-operation times, and $P^{\text{eff}}_{\text{shots}}$ is useful shot parallelism. The $\max(\cdot)$ term models the fact that an FTQC controller can pipeline logical gate execution and syndrome decoding; decoding is hidden only when it is no slower than the logical critical path. Host–QPU data movement and queue/control feedback remain serial in this model because hybrid loops must return measurement summaries, update parameters, or launch the next batch.

The recovery-cost factor $\alpha_R \geq 1$ couples quality recovery to runtime. The paper's default projection uses $\alpha_R = 1$ as a lower-bound requirement axis rather than a measured mitigation result. Real mechanisms such as deeper ansatz circuits, higher Trotter order, extra Pauli measurements, zero-noise extrapolation, or repeated noisy optimization would increase α_R, N_s, D_i, or N_{eval} and therefore move the frontier rightward.

The non-overlapped terms are decomposed rather than hidden:

$$T_{\text{serial}} = T_{\text{compile}} + T_{\text{map}} + T_{\text{route}} + T_{\text{ms}} + T_{\text{io}} + T_{\text{cq}} + T_{\text{queue}}.$$

Likewise, useful parallel shots are bounded by the slowest system resource:

$$P^{\text{eff}}_{\text{shots}} = \min(P_{\text{device}}, P_{\text{decode}}, P_{\text{control}}, P_{\text{queue}}, N_s N_{\text{eval}}).$$

This form is the hook for architecture studies: a surface-code stack can plug in code distance, cycle time, decoder bandwidth, and magic-state throughput, while a control-system study can change host-I/O and feedback terms. Existing resource-estimation tools already expose hardware assumptions such as QEC scheme, magic-state factories, physical-qubit counts, runtime, and Pareto frontiers [9], [16]. QArchGauge adds the application-side gate that those estimates alone do not answer: the measured native deadline and the measured quality gap for the same input.

Illustrative FT-stack parameters. The evaluation instantiates this interface with an intentionally optimistic, but physically anchored, surface-code-style stack. Surface-code and resource-estimation studies motivate exposing code distance, cycle time, logical-operation scheduling, magic-state throughput, decoder bandwidth, and control latency as first-class parameters [9], [10], [17], [18]. Recent superconducting surface-code experiments report a 1.1 μs error-correction cycle and 63 μs average real-time decoder latency at distance five [11]. Our default projection is not a measurement of such a deployed computer and is more optimistic: it uses $d = 25$, $\tau_c = 1\,\mu$s, $t_1 = d\tau_c$, $t_2 = 4d\tau_c$, $t_m = d\tau_c$, $P^{\text{eff}}_{\text{shots}} = 10^4$, a 5 μs pipelined decoder, 20 μs host–QPU I/O, and 30 μs queue/control feedback per circuit evaluation. These values are modeling knobs for sensitivity analysis, not claims about a quantum annealer or an existing gate-model QPU. A positive frontier under this default should be read as a lower-bound architecture target that must survive the conservative shot, host-I/O, queue/control, and recovery-cost sweeps in the evaluation.

Advantage frontier. QArchGauge inverts the model instead of assuming speedup. It asks what logical speed, shot throughput, overhead, and quality recovery are required for $T_{\text{qhw}} < T_{\text{native}}$ at the same quality. A case is speed-limited when quality is close enough but runtime remains large; quality-limited when faster gates do not help until the output gap closes; shot- or

TABLE II: Default projection knobs and sensitivity role.

Knob	Default value	Role in the projection
Code stack	$d = 25$, $\tau_c = 1\,\mu$s	Sets logical-operation scale; technology-specific studies can replace it with measured code distance, cycle time, and decoder bandwidth.
Logical operations	$t_1 = 25\,\mu$s, $t_2 = 100\,\mu$s, $t_m = 25\,\mu$s	Prices the measured 1Q, 2Q, and measurement depths; frontier plots vary effective execution improvement.
Pipeline and serial floor	$5\,\mu$s decode, $20\,\mu$s host I/O, $30\,\mu$s queue/control	Decode can overlap gate execution; host–QPU data movement and feedback remain serial. Figure 12b sweeps these bounds.
Shot parallelism	$P^{\text{eff}}_{\text{shots}} = 10^4$	Models useful shot lanes after device, decoder, control, and queue limits; sensitivity sweeps conservative, resource-estimator-like, LDPC/future-like, and aggressive batched scenarios.
Quality recovery	$R \in \{0, 0.5, 0.9, 1.0\}$, $\alpha_R \geq 1$	Requirement axis for closing measured quality gaps; real recovery mechanisms can increase shots, depth, evaluations, or mitigation overhead.

encoding-limited when repeated evaluations or data movement dominate; and native-dominated when a stronger classical path moves the break-even target.

Architecture target map. The projection output is intentionally phrased in architecture terms. If $D_2 t_2$ dominates and the single-lever target is finite, the case points to logical two-qubit latency, coupling, routing, or magic-state throughput. If $P^{\text{eff}}_{\text{shots}}$ is the active term, the useful hardware is not only more devices but enough parallel readout, decoding, control bandwidth, and queueing to keep those shots independent. If the decoder sets the pipeline critical path, the target is real-time decoding; if T_{io} or T_{queue} dominates, the target is near-QPU control, host–QPU data movement, measurement aggregation, and hybrid-loop scheduling. If the quality inequality fails, the target is representation, ansatz/search structure, fidelity, mitigation, or Trotter order before speed. If the circuit family is an Ising/QUBO energy-minimization instance, annealing-style specialization may be a candidate; otherwise the same record points to gate-model or analog/digital execution. This target map is the reason the evaluation reports both required speedup and where that speedup must come from.

E. Advantage Claim Checklist

Claim gates. QArchGauge reports a case as advantaged only after four checks hold in the same record. The native path must be the fastest valid implemented non-quantum candidate for the same input; the quantum-circuit path must use that input rather than an easier surrogate; the projected hardware terms must be named rather than hidden inside one speedup; and the output quality must satisfy the workload tolerance. If any check fails, the case remains useful evidence, but it is not an advantage claim. This checklist is what keeps simulator throughput, optimistic QPU projection, and application-level advantage from being confused.

F. Workload Suite

Records and audits. Every run emits JSON records with dataset metadata, model metadata, runtime breakdowns, quality, circuit qubits, depth, gate counts, shots, GPU metadata, and Slurm metadata where available. Summarizers validate case counts and write CSV/JSON artifacts used by the figures; audit scripts then compare manuscript claims against those artifacts. Login-node tests are correctness gates only, while performance evidence must come from completed Slurm jobs. This discipline also keeps the meaning of scaling explicit: the campaign may use hundreds of GPUs to collect cases, but the advantage test remains per same-input application case.

Failure handling. Failed paths remain measurement results. If a native candidate fails, the record keeps the error and continues with other candidates. If a quantum path fails, the record keeps available circuit metadata and marks the case as non-advantaged. This rule matters at scale because removing difficult cases would overstate the frontier.

Workload scope. The controlled calibration workload is `sklearn digits`. The practical suite covers AI classification, VQE-style molecular energy estimation, QAOA-style optimization, and Hamiltonian simulation. These are the application classes where users often expect quantum advantage, but every row follows the same comparison discipline: same input, strongest auditable native path, measured quantum-circuit path, and explicit quality target.

IV. Evaluation

This section uses a systems evaluation order: setup first, then the representative result, scale-out evidence, workload-specific quality analysis, component diagnosis, and architecture projection. The result is not a claim that quantum wins today. The result is a measured architecture model that shows how far each quantum-circuit application path is from its native HPC counterpart and which future-system resource must improve.

A. Evaluation Setup

Hardware Specification. Experiments run on a Top-20 TOP500 leadership-scale HPC system. Login-node smoke tests validate correctness, while performance experiments run through Slurm on GPU nodes with four NVIDIA A100 GPUs per node.

Benchmark. The unit of comparison is a same-input application case: QArchGauge measures a native HPC/ML path, measures a quantum-circuit application path on the simulator, records quality for both paths, and reports $T_{\text{qsim}}/T_{\text{native}}$ only when the quality target is interpretable. The completed campaign contains a 160-case controlled ML calibration, a 3,552-case practical suite, 116-case and 104-case Chem/Sim. coverage gates, weak scaling through 7,104 cases on 256 GPUs, direct fixed-work scaling from 64 to 256 GPUs, a four-workload larger stress gate, and 12 warmup-separated repeat trials.

Baselines. A case is one concrete input/configuration tuple, such as an ML class pair and circuit depth, a molecule and ansatz, a graph and QAOA setting, or a Hamiltonian and Trotter setting. The native path is never another quantum simulator; it is the fastest valid non-quantum candidate implemented for the same input and quality metric.

TABLE III: Workload setup, native target, quality gate, and recorded projection metadata. Counts are per same-input practical-suite case and report median/90th percentile; the ML row covers QNN/VQC, QKernel, and ML-Feature paths.

Workload	Native target	Quantum path	Quality gate	Cases and metadata
ML	scikit candidates plus PyTorch AMP/XGBoost gate	parameterized circuit, feature-map kernel, or quantum feature circuit	accuracy loss $\leq$0.02	160 calibration + 2,048 practical; features 4–10; eval. 224/320; 1Q 4,096/8,192; 2Q 1,760/3,584
Chem	dense exact or sparse Lanczos/eigensolver	VQE-style ansatz with Pauli-term energy aggregation	energy error $\leq$0.01	224 practical + 104 OpenFermion/PySCF; eval. 41/678; 1Q 444/2,712; 2Q 132/678
Opt.	exact, greedy, local-search, or simulated annealing	QAOA-style MaxCut circuit and repeated sampling	ratio loss $\leq$0.02	768 practical; nodes 4–7; eval. 101/169; 1Q 1,660.5/3,211; 2Q 1,210/2,366
Sim.	dense exact or sparse Krylov evolution	Trotterized Hamiltonian simulation circuit	observable error $\leq$0.01	512 practical; qubits 4–7; eval. 1/1; 1Q 140/492; 2Q 98/240

Feasibility. The suite is exact and state-vector-verifiable so every runtime, quality, and circuit-metadata record can be audited. It is a controlled architecture-modeling record, not a deployment-scale advantage claim. A one-node stress gate checks that the logging path extends beyond the smallest knobs: 512-sample 12-feature ML at depth 3, H_2O active-space Chem with an 8-qubit layer-3 VQE grid, 20-node MaxCut QAOA, and 12-qubit 16-step Heisenberg simulation. These stress cases are not folded into practical-suite medians. Measured plots use $T_{\text{qsim}}/T_{\text{native}}$, where `cuQuantum` executes the full circuit application today; projection plots use $T_{\text{qhw}}/T_{\text{native}}$, where Section III prices recorded gates, measurements, shots, repeated evaluations, and quality gaps.

B. Workload-Level Thresholds

To evaluate application-level measured thresholds, each point compares a quantum-circuit path with the selected native HPC/ML implementation on the same input and quality metric. This is not simulator-versus-simulator: the native side is the non-quantum path for the same case. Figure 4 shows the 3,552-case practical suite on 128 GPUs. The x-axis is the measured speedup that would be needed to replace today's `cuQuantum` application path with a future execution path at native time, the y-axis is quality gap to native, and outlined points show workload medians. Native paths finish in microseconds to milliseconds, while the simulated quantum-circuit paths take seconds per case; across all cases, the mean required speedup is 79,291$\times$ and the median is 15,164$\times$.

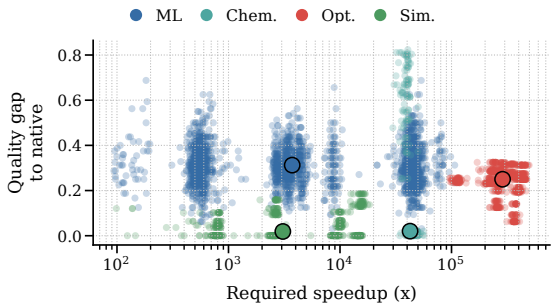


Fig. 4: Large practical-suite landscape over 3,552 cases. The plot keeps both axes that matter for advantage: required speedup and quality gap. Median required speedups are 3,726.4$\times$ for ML, 42,491.4$\times$ for Chem, 287,045.6$\times$ for Opt., and 3,071.0$\times$ for Sim.

ML. The native baseline is a classical classifier on the same feature vector and split: scikit-learn candidates in the main sweep plus a PyTorch AMP/XGBoost production-native gate. QKernel spends work on pairwise feature-map circuits to build a kernel matrix, while QNN/VQC trains a parameterized circuit. A QPU would make these circuit evaluations physical, but it would not remove feature loading, shots, kernel-matrix repetition, or optimizer feedback; ML therefore mixes runtime pressure with visible quality loss.

Chem. The native baseline uses the same active-space molecular Hamiltonian. Dense exact diagonalizes the full matrix; sparse Lanczos estimates the lowest energy through Hamiltonian-vector products. These solvers are strong at the executable qubit counts because they return deterministic energies without shots. The quantum path is VQE: prepare an ansatz, measure Pauli terms, and aggregate energy. A QPU can represent the molecular state physically, but the cases still pay two-qubit gates, repeated term measurements, and energy recovery.

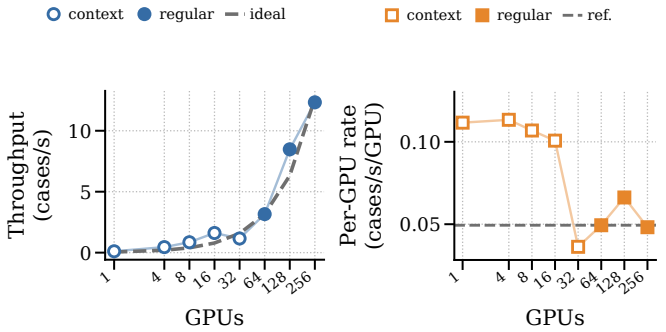
Opt. The optimization family uses MaxCut graphs. Exact search enumerates small graphs, while greedy, local-search, and simulated-annealing candidates improve cuts with cheap edge-count objectives. The quantum path is QAOA: edge weights become phase-separation gates, mixer gates explore bitstrings, and repeated sampling estimates the objective. A QPU could sample without state-vector simulation, but it still needs enough depth and parameter quality to beat cheap native heuristics.

Sim. The scientific-simulation family uses Hamiltonian time evolution. Dense exact methods materialize the Hamiltonian/evolution operator, while sparse Krylov methods use Hamiltonian-vector products. The quantum path is Trotterized Hamiltonian simulation, where local Hamiltonian terms become unitary gates. This is the most hardware-aligned mapping in the suite: a QPU applies unitary evolution instead of explicitly storing the full state vector. Sim. therefore has the lowest median threshold, although Trotter depth, measurements, and quality recovery still determine whether faster gates help.

Observation 1: The practical-suite gap is workload-specific. Native paths often finish in microseconds to milliseconds, while the simulated quantum-circuit path takes about seven seconds per case. The architecture question is therefore not whether the simulator is slow; it is whether future quantum systems should buy speed, quality recovery, shot parallelism, or lower control overhead for each workload family.

C. Scalability of the Evidence-Generation Framework

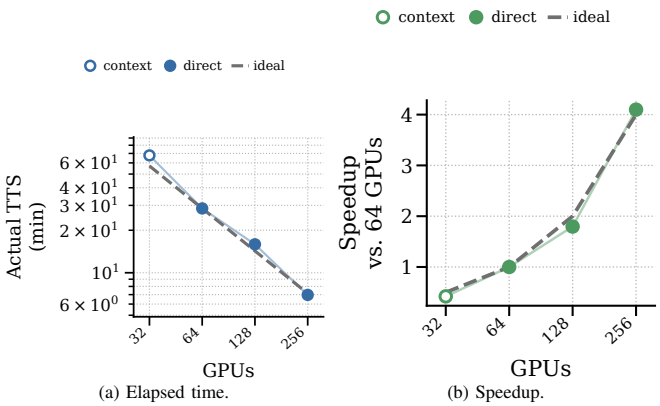
This subsection uses Perlmutter scaling to test measurement throughput, not to claim that a physical QPU scales like a GPU cluster. The large-profile suite contains independent same-input cases: weak scaling asks whether more GPUs can process more records, while strong scaling asks when a fixed evidence set loses useful per-GPU granularity. The QPU implication is separate: future accelerators need batched circuits, batched shots, repeated-evaluation scheduling, host–QPU I/O, and low-latency control.



(a) Total throughput. (b) Per-GPU stability.

Fig. 5: Weak scaling from 1 to 256 GPUs. The log GPU-count axis keeps the completed 1–32 GPU context ladder connected to the 64–256 GPU regular ladder instead of dropping the low-end runs. Hollow markers denote context runs with smaller per-GPU work; solid markers denote the regular large-profile weak-scaling regime. The right plot reports raw per-GPU rate, with the dashed line marking the regular-ladder median.

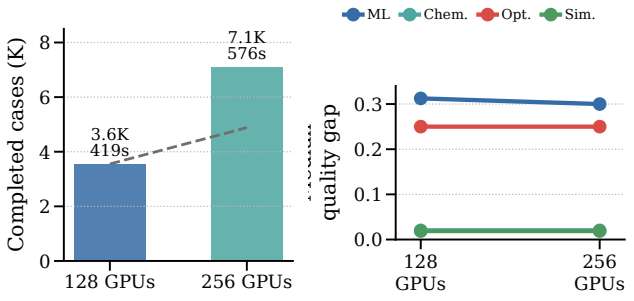
Weak scaling. Figure 5 keeps the completed ladder from 1 to 256 GPUs. The 1–32 GPU points are context runs that validate the low-end launch behavior, while the 64–256 GPU points form the regular weak-scaling regime where each GPU receives roughly 28 large-profile cases. The 64/128/256-GPU jobs completed with exit code 0:0 and generated 1,776, 3,552, and 7,104 ‘ok’ cases; the only nonempty stderr line was a benign NERSC monitoring warning. Throughput grows because cases are independent and per-GPU work remains large enough to amortize launch/orchestration. This builds a large evidence set; it does not mean that one circuit became faster by adding GPUs.



(a) Elapsed time. (b) Speedup.

Fig. 6: Strong scaling of the fixed 7,104-case application suite. The hollow 32-GPU marker is a completed 3,552-case context run normalized to the 7,104-case suite size; solid markers are direct fixed-suite regular-queue runs on 64, 128, and 256 GPUs. The direct run reduces time-to-solution from 28 minutes 33 seconds on 64 GPUs to 6 minutes 58 seconds on 256 GPUs.

Strong scaling. Figure 6 shows the fixed-work trend without hiding the lower end. The hollow 32-GPU point completed the proportional 3,552-case half suite in 33 minutes 54 seconds and is normalized to the 7,104-case size only for visual context; the solid 64/128/256-GPU points are direct fixed-work runs. Direct elapsed time falls from 28 minutes 33 seconds to 15 minutes 54 seconds and then 6 minutes 58 seconds. This is case-level parallelism, not distributed single-circuit execution: every Slurm task owns independent cases on one A100. The systems condition is granularity. At 256 GPUs the suite still provides roughly 28 cases/GPU, enough to amortize launch, Python orchestration, and task-local output; much smaller per-GPU work would again expose orchestration overhead.



(a) Work grows. (b) Quality remains stable.

Fig. 7: Larger-workload gate on 256 GPUs. The regular weak-scaling run completes 7,104 cases in 576 seconds with ‘ok’ status for every case. The larger run preserves the workload-level quality interpretation, so the extra cases increase evidence coverage rather than changing the advantage conclusion.

Workload growth and QPU batching requirement. Figure 7 shows the 7,104-case larger-workload gate. The 256-GPU weak-scaling run doubles the completed case count relative to 128 GPUs and finishes in 576 seconds, while workload-level quality gaps remain stable. More GPU workers increase evidence coverage but do not change the per-case advantage condition. The QPU lesson is batching: a future accelerator

needs enough independent circuits, shots, measurements, and hybrid-loop batches to amortize reset, host–QPU I/O, decoding, and control. If a workload is dominated by serial depth, measurement latency, optimizer feedback, or quality recovery, raw device count cannot move the advantage frontier by itself.

Observation 2: Scaling separates throughput from advantage. Independent circuit-application cases fill 256 GPUs, and the 7,104-case fixed suite still benefits from 64-to-256 GPU parallelism. For a QPU, the matching axis is not "more devices" alone; it is whether circuits, shots, and hybrid evaluations can be batched enough to amortize measurement, control, and feedback latency.

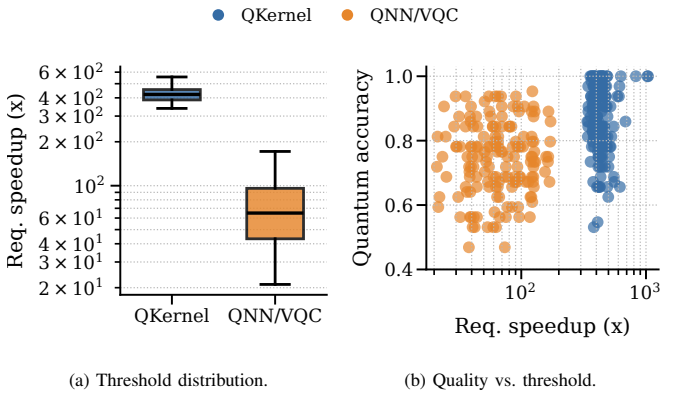
D. Application Quality Beyond Runtime

The workload-level landscape in Section IV-B reports runtime thresholds, but runtime alone is not quantum advantage. The `cuQuantum`/cuStateVec path is a state-vector execution of the chosen circuit application; the production suite does not inject a NISQ noise model. The quality gaps below are therefore application-quality gaps caused by feature encodings, ansatz/search limits, approximation order, optimizer behavior, and native-baseline strength.

ML model quality. Figure 8 shows the expanded `sklearn digits` calibration. The native path selects the fastest valid classifier on the same PCA features and split. QKernel maps those features into a quantum feature map and builds a kernel matrix from many circuit evaluations; QNN/VQC trains a parameterized circuit through a classical optimizer. This setup isolates an architecture-relevant issue: even with noiseless state-vector execution, the quantum model can miss native accuracy because the encoding or trainable circuit does not match the native model class.

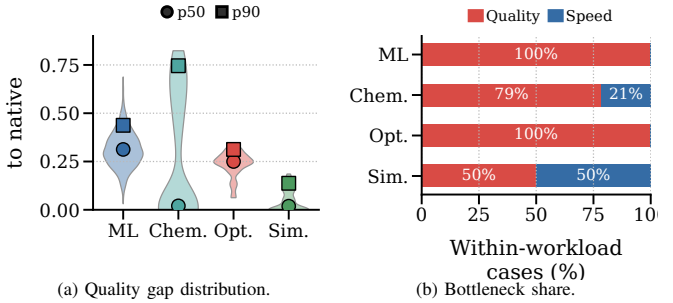
As shown in Figure 8, native ML accuracy ranges from 0.9219 to 1.0000. QKernel reaches median accuracy 0.8750 but needs 421.9× faster projected execution; QNN/VQC needs only 64.9× but drops to median accuracy 0.7500. The difference is model-specific rather than noise-specific: QKernel preserves more structure through a full kernel matrix but pays many circuit evaluations, while QNN/VQC is cheaper but more ansatz/optimizer sensitive. A 32-case production-style ML native gate confirms the baseline story: adding PyTorch AMP CNN/MLP and XGBoost changes the combined median threshold from 8,876.8× to 8,601.6×, while the production-only threshold is 49.3×. Profiling explains why this tiny-input native path is not GPU-kernel limited: GPU kernels occupy 0.8% of runtime, and Tensor-family kernels account for 15.2% of GPU-kernel time.

Workload quality bottlenecks. Figure 9 summarizes the 3,552-case practical suite. ML and Opt. are quality-limited in all measured cases, Chem has 176 quality-limited and 48 speed-limited cases, and Sim. splits evenly with 256 cases in each class. The quality-gap distribution explains why: ML has a median gap of 0.3125, Opt. has a median objective gap of 0.2500, Chem has a low median gap but a high tail, and Sim. has the smallest median gap but still retains quality-limited cases.



(a) Threshold distribution. (b) Quality vs. threshold.

Fig. 8: Expanded digits results over 160 cases. Both quantum-circuit paths are compared against the selected native HPC/ML baseline on the same inputs. The quantum-kernel path often reaches useful accuracy but needs hundreds of times faster execution, while QNN/VQC has a lower threshold but usually lower quality.

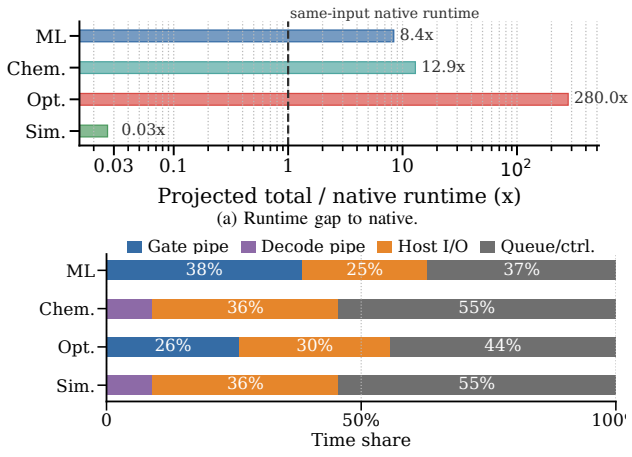


(a) Quality gap distribution. (b) Bottleneck share.

Fig. 9: Quality bottlenecks across the practical suite. Subfigure (a) shows the quality-gap distribution to native, with markers for median and 90th percentile. Subfigure (b) reports within-workload case fractions: a case is quality-limited when its quality gap exceeds tolerance; otherwise, the cases shown here are speed-limited when runtime remains the primary blocker.

These failures are not hardware-noise failures. Chem compares VQE-style energy aggregation against dense exact and sparse Lanczos/eigensolver baselines; Sim. compares Trotterized evolution against dense or sparse Krylov evolution; Opt. compares low-depth QAOA against exact, greedy, local-search, and simulated-annealing baselines. The Chem/Sim. accept gate completed 116 cases in 6 minutes and 5 seconds with median required speedups of 63,566.8× for Chem and 4,185.2× for Sim. The 104-case OpenFermion/PySCF Chem profile checks LiH and H_2O active-space fixtures; sparse Lanczos is the best-quality native method in 62 cases even when dense exact remains the selected runtime baseline.

Observation 3: The measured quality gaps are algorithmic and representational gaps, not injected NISQ noise. Across ML, Chem, Opt., and Sim., the quantum-circuit path must recover application quality: accuracy for ML, energy quality for Chem, objective quality for Opt., and evolution quality for Sim. This is why QArchGauge models advantage as a speed-quality frontier rather than a simulator-runtime ratio.



(a) Runtime gap to native.

(b) Projected time location.

Fig. 10: Projected runtime gap and time location at $P_{\text{shots}}^{\text{eff}} = 10^4$. Subfigure (a) shows the projected quantum time divided by the measured median runtime of the same-input native HPC/ML path; the dashed $1\times$ line means runtime parity with native. Subfigure (b) reports median within-workload time shares under the pipelined model: gate and decode compete for the critical pipeline stage, while host I/O and queue/control remain serial.

E. Architecture Target Diagnosis

The previous sections establish measured gaps and a complete evidence set. This subsection maps those records back to the design levers in Section III and diagnoses the first-order runtime target. The goal is not to label the simulator slow; it is to identify where a future quantum system spends time and whether one architecture lever can move the projected path to native-runtime parity.

Runtime gap and time location. Figure 10 applies the Section III execution model to every practical-suite case. Unlike Figure 4, which measures the full `cuQuantum` application path today, this view prices recorded metadata with the illustrative future stack. ML, Chem, and Opt. remain runtime-slower than native, with median projected totals of $8.9\times$, $13.3\times$, and $280.0\times$ native; Sim. is $0.03\times$ native because it has one median circuit evaluation and a direct Hamiltonian-evolution mapping. This is not current advantage: it still depends on the optimistic hardware stack and quality gate. The time-location plot shows why scalar speedup is insufficient. ML and Opt. have 224 and 101 median circuit evaluations; host I/O plus queue/control consumes 62% and 74% of median projected time. Chem has 41 evaluations and the clearest hybrid-control pressure: only 9% exposed decode-pipeline time after overlap, but 91% host I/O plus queue/control. Sim. has one median evaluation and 71% two-qubit share within its gate path, yet remains bounded by the serial launch/control floor at this scale.

Architecture pressure map. Figure 11 converts runtime gap and time location into an actionability map rather than one scalar QPU speedup. Each column names a first-order pressure: quality recovery, hybrid-loop count, decode pipeline, host–QPU I/O, queue/control feedback, two-qubit execution, or total projected runtime. The figure is not a current-hardware benchmark. Below-threshold logical memory and real-time decoding have been demonstrated only at small code distance, and industrial roadmaps still place large fault-tolerant machines later this decade [11], [19]. The targets are lower-bound design pressure under the optimistic Section III stack, not claims that an existing QPU, annealer, or cloud backend can execute these circuits at the shown rates. For projected quantum time T_{qhw} and quality-gap recovery R, a case enters the advantage region only if

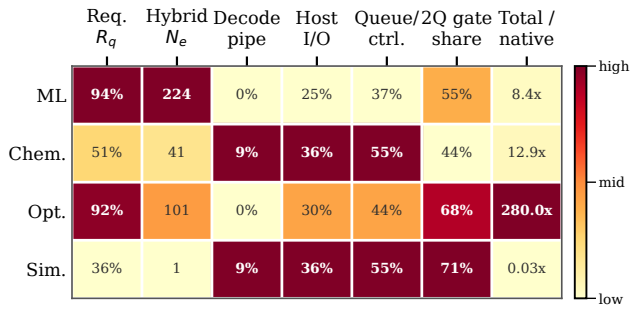
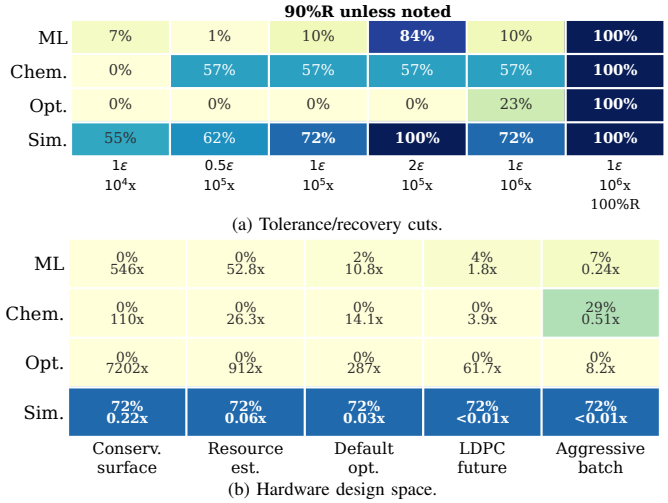


Fig. 11: Architecture pressure map at $P_{\text{shots}}^{\text{eff}} = 10^4$. Each cell reports a measured or projected target from the same case records: required quality-gap recovery R_q, median circuit evaluations N_e, exposed decode-pipeline share, host-I/O share, queue/control share, two-qubit share within the logical gate path, and median projected total time relative to the native runtime. Darker cells mark stronger within-column pressure.

$$T_{\text{qhw}} < T_{\text{native}} \quad \text{and} \quad (1-R)\Delta Q \le \epsilon_w,$$

where ΔQ is the measured quality gap and ϵ_w is the workload tolerance. The recovery parameter R is a requirement axis, not a measured mitigation result; any real mechanism that raises R can also increase shots, depth, or evaluations through α_R. The map shows why the target is architectural. ML has high required recovery and high hybrid count, so useful hardware needs batched submission and low-latency feedback only after the representation improves. Chem exposes host-I/O and queue/control pressure from repeated energy evaluations, making near-QPU control, measurement aggregation, and observable batching first-order. Opt. combines high recovery, many hybrid evaluations, queue/control pressure, and two-qubit share, so speed alone cannot compensate for shallow search. Sim. has low hybrid count and low runtime ratio under the optimistic model, but its gate-path pressure moves to two-qubit execution and Trotter/observable quality. Quantum annealing is relevant mainly to optimization problems embeddable as energy minimization [20]; it is not a drop-in replacement for QNN/QKernel, VQE chemistry, or Trotterized Hamiltonian simulation.

Observation 4: Architecture targets require amount, location, and actionability. Figure 10a shows whether runtime is a barrier, Figure 10b shows where projected time is spent after gate/decode overlap, and Figure 11 maps the dominant design pressure. The useful accelerator is workload-specific: representation plus hybrid-loop co-design for ML and Opt., near-QPU I/O/control and observable batching for Chem, and Hamiltonian-simulation-oriented execution for Sim.; annealing is only a candidate for the optimization subset, not a general substitute for the gate-model paths.



(a) Tolerance/recovery cuts.

(b) Hardware design space.

Fig. 12: Sensitivity maps beyond the component target. Subfigure (a) sweeps selected speed, quality-recovery, and workload-tolerance cuts; cells report advantaged-case fractions and use the base tolerance ϵ_w unless the column states 0.5ϵ or 2ϵ. Subfigure (b) is a no-srun projection sweep over conservative surface-code, resource-estimator-like, default optimistic, LDPC/future-like, and aggressive batched hardware scenarios. Cells report advantaged-case fraction and median $T_{\text{qhw}}/T_{\text{native}}$ at 90% quality-gap recovery under the lower-bound $\alpha_R = 1$ recovery-cost assumption.

F. Sensitivity and Design Frontier

Runtime parity is necessary but not sufficient for quantum advantage. A QPU can beat native time and still lose if output quality misses tolerance; a quality-correct circuit can also remain unusable if the FT stack leaves a serial host/control floor. Quality recovery is not free: richer ML encodings, deeper VQE/QAOA circuits, more Pauli measurements, or higher Trotter order can change both ΔQ and runtime. Figure 12 checks the two assumptions behind Figures 10 and 11: application recovery and hardware-stack scaling.

Figure 12a isolates the application side. At 90% recovery and the base tolerance, Sim. reaches 55% of cases at $10^4\times$ and 72% at $10^5\times$, Chem reaches 57% at $10^5\times$, and Opt. remains at 23% even at $10^6\times$. Tightening the tolerance to $0.5\epsilon_w$ leaves Chem unchanged in this cut but reduces Sim. to 62.5%; relaxing to $2\epsilon_w$ moves ML from 9.8% to 83.8% and Sim. to 100% at $10^5\times$. These are projected lower-bound wins, not current-hardware wins: the figure treats R and ϵ_w as requirements and does not grant a free mitigation mechanism. If recovery requires zero-noise extrapolation, deeper ansatz/search, higher Trotter order, or more Pauli samples, α_R grows and the case moves to a larger speed/shot/host-control budget.

Figure 12b isolates the hardware-stack side with an offline sweep over conservative surface-code, resource-estimator-like, default optimistic, LDPC/future-like, and aggressive batched scenarios [9]–[12]. Conservative and resource-estimator-like assumptions erase most optimistic coverage; the default stack opens Sim. and a small ML slice; LDPC/future-like and aggressive batching reduce ML/Chem runtime but leave Opt. quality-limited. Because the sweep separates decode from host-I/O and queue/control floors, the speedup source is not a scalar QPU clock. ML and Opt. need representation and hybrid-loop co-design before more shot lanes help. Chem is the clearest near-QPU control target; Sim. points to Hamiltonian-simulation-oriented analog/digital execution. Annealing can matter for embeddable optimization instances, but it is not a general replacement for gate-model ML, Chem, or Sim.

Observation 5: Sensitivity separates application tolerance/recovery from hardware scaling. The tolerance map shows which workloads depend on the application-quality contract before speed matters; the hardware map shows that conservative shot bounds remove much of the optimistic frontier. Chem is the clearest near-QPU control target, Opt. needs broader co-design, and Sim. is a runtime-ready but quality-gated accelerator candidate only under explicit future-hardware assumptions.

V. Discussion and Future Work

Architecture Meaning of the Frontier: QArchGauge does not prove current quantum advantage. It turns advantage into a computer-architecture boundary: once task, quality target, and native baseline are fixed, the remaining gap is assigned to logical execution, shot parallelism, decoder/control overhead, native-baseline strength, or quantum algorithm quality. The output is a workload-specific target for what a future QPU must change before the full quantum-circuit application beats the native HPC path.

Future QPU Control Stack: The main implication is that a single "faster quantum computer" roadmap is too coarse. Surface-code demonstrations emphasize decoding and cycle time [11], LDPC proposals trade qubit overhead against connectivity and decoder requirements [12], and FTQC roadmaps include modularity, magic-state resources, and control hardware [19]. Our results map those choices to applications: ML and Opt. need quality-preserving encodings, batched submission, and low-latency feedback; Chem exposes decoder bandwidth, measurement aggregation, and observable batching; Sim. points toward Hamiltonian-simulation-oriented analog/digital execution. Annealing can help embedded optimization cases [20], but it is not a general replacement for the gate-model paths measured here.

HPC–Quantum Integrated Architecture: QHPC systems should treat a QPU as an accelerator inside a scientific workflow, not as a detached queue. QArchGauge supplies the quantitative contract such systems need: native deadlines, quality tolerances, circuit-evaluation counts, shot-parallel regions, and serial host/control floors. These signals let an integrated runtime co-schedule CPU/GPU kernels, simulator backends, QPU batches, measurements, optimizers, and quality checks.

Artifact Availability and Reproducibility: The artifact is designed for HPCA-style re-analysis, not only figure reproduction. The release package will include anonymized JSON/CSV summaries, Slurm scripts, figure/projection scripts, audits, and rebuild instructions. Each record exposes native deadlines, circuit metadata, quality gaps, hardware knobs, and job provenance, so reviewers can change code distance, shot lanes,

TABLE IV: Deployment-scale proxy boundary. Proxies extend the record format without claiming full state-vector execution for every larger case.

Workload	Proxy record
ML	Anchor: 512-sample, 12-feature depth-3 stress case with PyTorch/XGBoost native gate. Scale axis: larger samples/features, richer kernels, deeper trainable circuits. Signal: hybrid count, feature loading, batch/feedback latency, native deadline shift.
Chem	Anchor: H_2O 8-qubit layer-3 VQE stress case and 104 OpenFermion/PySCF cases. Scale axis: larger active spaces, Pauli grouping, richer ansatz families. Signal: energy gap, Pauli evaluations, decoder/-control and observable batching.
Opt.	Anchor: 20-node MaxCut QAOA stress case and small-graph practical suite. Scale axis: 50–100 node graphs, tuned heuristics/MILP, QAOA metadata. Signal: search budget, circuit count, objective gap, annealing or gate-model fit.
Sim.	Anchor: 12-qubit 16-step Heisenberg stress case and Trotter suite. Scale axis: 16–20 qubit sparse/tensor checks and deeper Trotter ladders. Signal: 2Q depth, observable quality, Krylov/tensor deadline, analog/digital fit.

host-I/O floors, native baselines, or recovery-cost multipliers without repeating the GPU campaign.

Deployment-Scale Proxy Boundary: Exact state-vector execution cannot be pushed to deployment-scale molecules, graphs, or Hamiltonians by simply spending more GPU node-hours. The next step is a proxy record: keep the native deadline, circuit metadata, quality target, and projection knobs, while replacing full state-vector execution with domain solvers, tensor/approximate checks, or metadata-only circuit construction when exact simulation becomes the bottleneck. Table IV states this boundary; each row is a proxy record, not a deployment-scale state-vector claim.

Scope and Future Work: The largest modeling risk is an underpowered native baseline. Stronger natives on larger tasks–deeper CNN/ResNet families, boosted trees, tuned MILP/metaheuristics, tensor networks, projector QMC, and CCSD(T)-class references–can still move the frontier rightward; the method quantifies how much. `cuQuantum` is instrumentation, not the future hardware stack. The current quality gaps are noiseless circuit-application gaps, so hardware noise, SPAM error, shot noise, barren plateaus, and noisy optimizer convergence would generally increase the recovery cost. Hardware-specific studies should replace the default pipeline and serial terms with measured code distance, cycle time, logical-error budget, magic-state throughput, decoder latency, host-I/O bandwidth, controller memory movement, and device concurrency. Future versions should attach energy and mitigation cost by comparing $E_{\mathrm{native}} = N_{\mathrm{GPU}} P_{\mathrm{GPU}} T_{\mathrm{native}}$ with refrigerator, decoder, control, host, and error-mitigation overhead over T_{qhw}. The bundled Slurm runs, 12 warmup-separated trials, zero failures, maximum quantum-runtime CV is 0.0400, and audits reduce timing and provenance risk.

VI. Related Work

Quantum Simulation and NISQ Systems. Computer architecture uses measurement and modeling to study immature systems [21]; QArchGauge applies that discipline to quantum advantage. ScaleQsim, AURORA-Q, and `cuQuantum` improve state-vector simulation through distribution, GPUs, tiered memory, asynchronous movement, and optimized primitives [4]–[6]. Benchmark suites such as SuperMarQ, QASMBench, MQT Bench, QAOAKit, QED-C, and QCHALLenge provide circuits, metrics, parameters, and application-aware tests [2], [3], [7], [8], [22], [23]. These works are essential measurement substrates, but they do not by themselves compare the full quantum-circuit application path with a same-input native HPC deadline and quality target.

Quantum Applications and Native Baselines. Quantum kernels, variational classifiers, VQE chemistry, QAOA optimization, and Hamiltonian simulation are common advantage candidates [24]–[29]. Each domain also has strong native baselines, from classical ML libraries to exact, sparse, heuristic, and Krylov-style solvers [15]. Fault-tolerant and resource-estimation work models code distance, cycle time, layout, magic-state factories, decoder latency, and physical assumptions [9]–[11], [17], [18]; QHPC and hybrid-runtime studies add gateway interfaces, queueing, fidelity, scheduling, and classical-resource constraints [13], [14], [30]–[33]. QArchGauge is complementary: it supplies same-task native baselines, measured quality gaps, repeated-evaluation counts, circuit metadata, and shot, host-I/O, and queue/control sensitivity artifacts that those estimators and runtimes can plug into.

VII. Conclusion

In this paper, we present QArchGauge for same-input, same-quality quantum-advantage modeling. The evaluation follows five gates: define the native and quantum application paths, measure workload-level thresholds, scale evidence collection on Perlmutter, test application quality beyond runtime, and project the result onto architecture levers. The controlled digits sweep requires 421.9× for quantum kernels and 64.9× for QNN/VQC; the practical suite contains 3,552 cases and shows measured simulator-path thresholds from 3,071.0× for Sim. to 287,045.6× for Opt. Under the default lower-bound projection at 90% quality-gap recovery, $10^4\times$ covers 54.9% of Sim. cases and $10^5\times$ covers 57.1% of Chem cases, while conservative shot, host-I/O, queue/control, and recovery-cost assumptions remove much of that coverage and ML/Opt. remain quality-limited. The main lesson is to report an advantage frontier and architecture target map, not a yes/no slogan.

References

[1] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[2] T. Tomesh, P. Gokhale, V. Omole, G. S. Ravi, K. N. Smith, J. Viszlai, X.-C. Wu, N. Hardavellas, M. R. Martonosi, and F. T. Chong, "SupermarQ: A scalable quantum benchmark suite," in *2022 IEEE International Symposium on High-Performance Computer Architecture*, 2022, pp. 587–603.

[3] T. Lubinski, S. Johri, P. Varosy, J. Coleman, L. Zhao, J. Necaise, C. H. Baldwin, K. Mayer, and T. Proctor, "Application-oriented performance benchmarks for quantum computing," *IEEE Transactions on Quantum Engineering*, vol. 4, pp. 1–32, 2023.

[4] NVIDIA, "NVIDIA cuQuantum SDK Documentation," https://docs.nvidia.com/cuda/cuquantum/, 2026, accessed July 9, 2026.

[5] C. Kim, E. Sohn, S. Kim, A. Sim, K. Wu, H. Tang, Y. Son, and S. Kim, "ScaleQsim: Highly Scalable Quantum Circuit Simulation Framework for Exascale HPC Systems," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 9, no. 3, Dec. 2025. [Online]. Available: https://doi.org/10.1145/3771577

[6] C. Kim, A. Sim, K. Wu, H. Tang, Y. Son, J. Park, and S. Kim, "AURORA-Q: Asynchronous Unified Resource Optimizer for Quantum Simulation on HPC System," Accepted to the IEEE International Conference on Distributed Computing Systems, 2026, accepted for publication.

[7] A. Li, S. Stein, S. Krishnamoorthy, and J. Ang, "QASMBench: A low-level quantum benchmark suite for NISQ evaluation and simulation," *ACM Transactions on Quantum Computing*, vol. 4, no. 2, pp. 1–26, 2023.

[8] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.

[9] W. van Dam, M. Mykhailova, and M. Soeken, "Using azure quantum resource estimator for assessing performance of fault tolerant quantum computation," 2023.

[10] M. Webber, V. Elfving, S. Weidt, and W. K. Hensinger, "The impact of hardware specifications on reaching quantum advantage in the fault tolerant regime," *AVS Quantum Science*, vol. 4, no. 1, p. 013801, 2022.

[11] Google Quantum AI and Collaborators, "Quantum error correction below the surface code threshold," *Nature*, vol. 638, pp. 920–926, 2025.

[12] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, and T. J. Yoder, "High-threshold and low-overhead fault-tolerant quantum memory," *Nature*, vol. 627, pp. 778–782, 2024.

[13] T. Beck, A. Baroni, R. Bennink, G. Buchs, E. A. Coello Pérez, M. Eisenbach, R. Ferreira da Silva, M. Gopalakrishnan Meena, K. Gottiparthi, P. Groszkowski, T. S. Humble, R. Landfield, K. Maheshwari, S. Oral, M. A. Sandoval, A. Shehata, I.-S. Suh, and C. Zimmer, "Integrating quantum computing resources into scientific HPC ecosystems," 2024.

[14] S. Chundury, A. Shehata, S. Kim, M. Gopalakrishnan Meena, C. Lu, K. Gottiparthi, E. A. Coello Perez, F. Mueller, and I.-S. Suh, "Scaling hybrid quantum–HPC applications with the quantum framework," in *SC Workshops 2025: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 1888–1897.

[15] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[16] Microsoft Azure Quantum, "Introduction to the Resource Estimator," https://learn.microsoft.com/en-us/azure/quantum/intro-to-resource-estimation, 2026, accessed July 9, 2026.

[17] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, "Surface codes: Towards practical large-scale quantum computation," *Physical Review A*, vol. 86, no. 3, p. 032324, 2012.

[18] D. Litinski, "A game of surface codes: Large-scale quantum computing with lattice surgery," *Quantum*, vol. 3, p. 128, 2019.

[19] IBM Quantum, "IBM Lays Out Clear Path to Fault-Tolerant Quantum Computing," https://www.ibm.com/quantum/blog/large-scale-ftqc, 2025, accessed July 9, 2026.

[20] D-Wave Quantum Inc., "What Is Quantum Annealing?" https://docs.dwavequantum.com/en/latest/quantum_research/quantum_annealing_intro.html, 2026, accessed July 9, 2026.

[21] K. Skadron, M. Martonosi, D. I. August, M. D. Hill, D. J. Lilja, and V. S. Pai, "Challenges in computer architecture evaluation," *Computer*, vol. 36, no. 8, pp. 30–36, 2003.

[22] R. Shaydulin, K. Marwaha, J. Wurtz, and P. C. Lotshaw, "QAOAKit: A toolkit for reproducible study, application, and verification of the QAOA," in *2021 IEEE/ACM Second International Workshop on Quantum Computing Software*, 2021, pp. 64–71.

[23] M. Erdmann, L. Karch, A. Awasthi, C. I. Jones, P. Bhardwaj, F. Krellner, J. Stein, C. Linnhoff-Popien, N. Kraus, J. P. Eder, S. Braun, and T. Liu, "Quantum computing—strategic recommendations for the industry," 2026.

[24] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, "Quantum machine learning," *Nature*, vol. 549, no. 7671, pp. 195–202, 2017.

[25] M. Cerezo, G. Verdon, H.-Y. Huang, L. Cincio, and P. J. Coles, "Challenges and opportunities in quantum machine learning," *Nature Computational Science*, vol. 2, no. 9, pp. 567–576, 2022.

[26] A. Kandala, A. Mezzacapo, K. Temme, M. Takita, M. Brink, J. M. Chow, and J. M. Gambetta, "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets," *Nature*, vol. 549, no. 7671, pp. 242–246, 2017.

[27] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," 2014.

[28] I. M. Georgescu, S. Ashhab, and F. Nori, "Quantum simulation," *Reviews of Modern Physics*, vol. 86, no. 1, pp. 153–185, 2014.

[29] A. J. Daley, I. Bloch, C. Kokail, S. Flannigan, N. Pearson, M. Troyer, and P. Zoller, "Practical quantum advantage in quantum simulation," *Nature*, vol. 607, no. 7920, pp. 667–676, 2022.

[30] A. Shehata, P. Groszkowski, T. Naughton, M. Gopalakrishnan Meena, E. Wong, D. Claudino, R. Ferreira da Silva, and T. Beck, "Bridging paradigms: Designing for HPC-quantum convergence," *Future Generation Computer Systems*, vol. 174, p. 107980, 2026.

[31] D. Camps, E. Rrapaj, K. Klymko, H. Kim, K. Gott, S. Darbha, J. Balewski, B. Austin, and N. J. Wright, "Quantum computing technology roadmaps and capability assessment for scientific computing: An analysis of use cases from the NERSC workload," Lawrence Berkeley National Laboratory, Tech. Rep., 2025.

[32] S. Pehlivanoglu, U. de Muelenaere, P. Kogge, and A. Sabry, "Qurator: Scheduling hybrid quantum-classical workflows across heterogeneous cloud providers," 2026.

[33] M. Tejedor, M. Grossi, C. Tüysüz, R. Rocha, and S. Vallecorsa, "Kubernetes-orchestrated hybrid quantum-classical workflows," 2026.

AI USE

OpenAI Codex assisted with drafting, editing, code generation, artifact consistency checks, and LaTeX build debugging. The authors are responsible for the final technical content, experiments, figures, citations, and claims.